Blueprint: Coefficient choice in Galois-ring lifts of APN permutations

Hiroki Fukui

Version 2.0.0, 27 October 2026

Purpose

This blueprint maps every result of the paper *Coefficient choice in Galois-ring lifts of APN permutations: sharp unit-direction spectra and one-term optimal lifts* to the Lean 4 declarations that formalize it, states what remains on paper, and names the finite computations that cross-check it. Numbers refer to the paper (`paper/manuscript.pdf`). The Lean development is in `supplement/lean` (Lean 4.31.0-rc1, Mathlib `d568c8c`): 55 modules, the root `Apn2R03.lean` and `Audit.lean`. `Audit.lean` prints the axioms of 150 declarations; in the author's clean build every one depends only on `propext`, `Classical.choice` and `Quot.sound`, and no declaration uses `sorry`. This blueprint was generated by `blueprint/gen_blueprint.py`, which checks that every declaration named below is audited in `Audit.lean` and defined in exactly one module.

Settings. Most results are proved for an abstract finite ring R with a surjection onto a finite field K whose kernel is $2R$, with $2^k = 0$ and 2-torsion $2^{k-1}R$; the declarations `grs`, `grs_card` and `exists_GR_poly` show that $\mathrm{GR}(2^k, m)$ satisfies these axioms, and the declarations ending in `_GR` state results on the concrete ring. Difference counts are cardinalities of actual finite solution sets.

Status. 22 nodes; "formalized" means that every mathematical claim of the item is a Lean theorem; otherwise the paper-only parts are listed.

Paper	Item	Status
Lemma 4.1	Lift invariants A_F and E_F are well defined; their control	formalized
Lemma 3.2	Hypothesis (H) gives the abstract hypotheses used in Lean	formalized
Section 3	Galois rings satisfy the abstract axioms used in Lean	formalized
Theorem B	Minimax over all lifts	formalized
Theorem C	All minimizers	formalized
Theorem D	Exceptional spectra and certification modulo 8	formalized
Corollary E	Optimality across levels (and Theorem 6.3, Corollary 6.4)	formalized
Theorem F	Penultimate layer, middle layer at $k = 3$, and the trade-off	formalized

Paper	Item	Status
Theorem 8.4	Intermediate layers (Theorem 8.4) and Corollary 8.5	formalized
Proposition 7.1	The residue $P_{m,r}$ is an APN permutation	formalized
Theorem A	A one-term optimal lift of $P_{m,r}$	core formalized; parts on paper
Corollary 7.5	The natural lift at $k = 3$	formalized
Proposition 9.1	Valuation preservation	formalized
Proposition 9.3	Markov property and product bound for characteristics (Corollary G (a))	formalized
Proposition 9.4	Characteristics in every 2-adic layer (Corollary G (b))	formalized
Proposition 9.5	Two-round differentials for every lift (Corollary G (c), first part)	core formalized; parts on paper
Proposition 9.6	Layerwise two-round differential bounds without a support factor (Corollary G (c), second part)	core formalized; parts on paper
Proposition 9.7	Fixed keys (Corollary G (d))	formalized
Proposition 9.8	Residue partition	core formalized; parts on paper
Theorem 10.1	Representatives with $\mu = 0$ and $Z = \emptyset$ for every odd m	core formalized; parts on paper
Theorem 10.2	A sparse family (at most 16 monomials)	core formalized; parts on paper
Proposition 10.3	Low-cost optimal lifts by one coefficient digit	formalized for the family $P_{m,r}$; general case on paper

Lemma 4.1 (p. 7): Lift invariants A_F and E_F are well defined; their control

Status: formalized.

Lean:

- `APN2.GRS.A_spec` (`Apn2R03/Invariants.lean`)
- `APN2.GRS.E_spec` (`Apn2R03/Invariants.lean`)
- `APN2.GRS.E_transl` (`Apn2R03/Invariants.lean`)
- `APN2.GRS.A_prescribe` (`Apn2R03/Control.lean`)
- `APN2.GRS.E_prescribe` (`Apn2R03/Control.lean`)
- `APN2.GRS.A_control` (`Apn2R03/Control.lean`)
- `APN2.GRS.E_control` (`Apn2R03/Control.lean`)

Lemma 3.2 (p. 6): Hypothesis (H) gives the abstract hypotheses used in Lean

Status: formalized.

Lean:

- `APN2.GRS.hyp_of_H` (`Apn2R03/HypFromH.lean`)

Section 3 (p. 6): Galois rings satisfy the abstract axioms used in Lean

Status: formalized.

Lean:

- `APN2.grs` (`Apn2R03/GRInst.lean`)
- `APN2.grs_card` (`Apn2R03/GRInst.lean`)
- `APN2.exists_GR_poly` (`Apn2R03/MainGR.lean`)
- `APN2.zmod8` (`Apn2R03/Basic.lean`)

Theorem B (p. 4): Minimax over all lifts

Status: formalized.

Lean:

- `APN2.GRS.minimax_k3_attain` (`Apn2R03/MainK3.lean`)
- `APN2.GRS.minimax_k4_attain` (`Apn2R03/MainK4.lean`)
- `APN2.GRS.minimax_k3_lower` (`Apn2R03/MainK3.lean`)
- `APN2.GRS.minimax_k4_lower` (`Apn2R03/MainK4.lean`)

Computational cross-check: validate_v3 at $m = 5$ (32 records: 27 MATCH, 5 BOUND_ONLY).

Uses: Lemma 4.1, Lemma 3.2.

Theorem C (p. 5): All minimizers

Status: formalized.

Lean:

- `APN2.GRS.minimizers_k3` (`Apn2R03/MainK3.lean`)
- `APN2.GRS.minimizers_k4` (`Apn2R03/MainK4.lean`)

Computational cross-check: validate_v3.

Uses: Theorem B.

Theorem D (p. 5): Exceptional spectra and certification modulo 8

Status: formalized.

Lean:

- `APN2.GRS.spectrum_mu0` (`Apn2R03/Spectrum3.lean`)
- `APN2.GRS.spectrum_mu` (`Apn2R03/Spectrum2.lean`)
- `APN2.GRS.spectrum_k3` (`Apn2R03/Spectrum4.lean`)
- `APN2.GRS.delta_stable_mod8` (`Apn2R03/StabilityLit.lean`)
- `APN2.delta_stable_mod8_GR` (`Apn2R03/StabilityLit.lean`)

Computational cross-check: validate_v3, e_tests.

Uses: Lemma 4.1.

Corollary E (p. 5): Optimality across levels (and Theorem 6.3, Corollary 6.4)

Status: formalized.

Lean:

- `APN2.GRS.cross_level_stay` (`Apn2R03/CrossLevel.lean`)
- `APN2.GRS.cross_level_exists` (`Apn2R03/CrossLevel.lean`)
- `APN2.cross_level_stay_GR` (`Apn2R03/GRCompat.lean`)
- `APN2.cross_level_exists_GR` (`Apn2R03/GRCompat.lean`)
- `APN2.GRS.coherent_family` (`Apn2R03/Coherent.lean`)
- `APN2.coherent_family_GR` (`Apn2R03/GRTower.lean`)
- `APN2.towerGR` (`Apn2R03/GRTower.lean`)

Computational cross-check: tests/test_cross_level.py.

Uses: Theorem C, Theorem D.

Theorem F (p. 5): Penultimate layer, middle layer at $k = 3$, and the trade-off

Status: formalized.

Lean:

- `APN2.GRS.penultimate` (`Apn2R03/Layers.lean`)
- `APN2.GRS.middle_k3` (`Apn2R03/Layers.lean`)
- `APN2.GRS.O3_iff` (`Apn2R03/Layers.lean`)
- `APN2.GRS.tradeoff` (`Apn2R03/Layers.lean`)

Computational cross-check: layers/.

Uses: Lemma 4.1.

Theorem 8.4 (p. 14): Intermediate layers (Theorem 8.4) and Corollary 8.5

Status: formalized.

Lean:

- `APN2.GRS.inter_dvd` (`Apn2R03/Inter.lean`)
- `APN2.GRS.inter_upper` (`Apn2R03/Inter3.lean`)
- `APN2.GRS.inter_row` (`Apn2R03/Inter5.lean`)
- `APN2.GRS.inter_count` (`Apn2R03/Inter5.lean`)
- `APN2.GRS.inter_mult` (`Apn2R03/InterMult.lean`)
- `APN2.GRS.inter_flat_iff` (`Apn2R03/Inter4.lean`)
- `APN2.GRS.inter_layer_iff` (`Apn2R03/InterMult.lean`)
- `APN2.GRS.fam5_interfamily` (`Apn2R03/InterFam.lean`)
- `APN2.GRS.fam5_nat_inter` (`Apn2R03/InterMult.lean`)

On paper only: Proposition 8.6 (digit recursion) and the classification of the remaining configurations.

Computational cross-check: layers/r05c (133 configurations, 75 full rows, 691 earlier rows).

Uses: Theorem F.

Proposition 7.1 (p. 11): The residue $P_{m,r}$ is an APN permutation

Status: formalized.

Lean:

- `APN2.resAPN3` (`Apn2R03/ResAPN.lean`)
- `APN2.resAPN5` (`Apn2R03/ResAPN.lean`)
- `APN2.resPerm3` (`Apn2R03/ResPerm.lean`)
- `APN2.resPerm5` (`Apn2R03/ResPerm.lean`)
- `APN2.cube_inj` (`Apn2R03/ResPerm.lean`)

Theorem A (p. 4): A one-term optimal lift of $P_{m,r}$

Status: core formalized; parts on paper.

Lean:

- `APN2.GRS.fam3_units'` (`Apn2R03/ResAPN.lean`)
- `APN2.GRS.fam5_units'` (`Apn2R03/ResAPN.lean`)
- `APN2.thmA3_GR` (`Apn2R03/MainGR.lean`)
- `APN2.thmA3_GR_units` (`Apn2R03/MainGR2.lean`)
- `APN2.thmA5_GR` (`Apn2R03/MainGR.lean`)
- `APN2.thmA5_GR_units` (`Apn2R03/MainGR2.lean`)
- `APN2.GRS.fam3_lower_all` (`Apn2R03/Family3c.lean`)
- `APN2.GRS.fam5_lower_all` (`Apn2R03/Family5b.lean`)
- `APN2.GRS.fam3_spectrum` (`Apn2R03/Family3c.lean`)
- `APN2.GRS.fam5_spectrum` (`Apn2R03/Family5b.lean`)
- `APN2.natDegree_Fsh3` (`Apn2R03/Degree.lean`)
- `APN2.natDegree_Fsh5` (`Apn2R03/Degree.lean`)
- `APN2.GRS.fam3_natural_exact` (`Apn2R03/Natural.lean`)
- `APN2.GRS.fam5_natural_q2` (`Apn2R03/Natural.lean`)
- `APN2.GRS.fam5_nat_k3` (`Apn2R03/Natural.lean`)
- `APN2.GRS.fam5_nat_k3_spectrum` (`Apn2R03/Natural2.lean`)

On paper only: the operation count in the termwise evaluation model; which monomials occur.

Computational cross-check: validate_v3 ($m = 5$ all rows; $m = 7$, $P_{7,3}$ and $P_{7,5}$, all 16384 rows); Magma, 64 directions.

Uses: Theorem B, Theorem D, Proposition 7.1.

Corollary 7.5 (p. 12): The natural lift at $k = 3$

Status: formalized.

Lean:

- `APN2.GRS.fam5_nat_k3` (`Apn2R03/Natural.lean`)

- `APN2.GRS.fam5_nat_k3_spectrum` (`Apn2R03/Natural2.lean`)
- `APN2.A5n_zero` (`Apn2R03/Natural.lean`)
- `APN2.GRS.fam5_nat_E0` (`Apn2R03/Natural.lean`)
- `APN2.GRS.cc_A0_mu0_k3` (`Apn2R03/Natural.lean`)

Computational cross-check: $P_{7,5}$ all rows; $P_{9,5}$, $P_{9,7}$ sampled rows.
Uses: Theorem D, Proposition 7.1.

Proposition 9.1 (p. 15): Valuation preservation

Status: formalized.
Lean:

- `APN2.GRS.unit_diff` (`Apn2R03/Ordinary.lean`)
- `APN2.GRS.nonunit_diff` (`Apn2R03/Ordinary.lean`)

Proposition 9.3 (p. 16): Markov property and product bound for characteristics (Corollary G (a))

Status: formalized.
Lean:

- `APN2.Markov.trail_card` (`Apn2R03/Markov.lean`)
- `APN2.Markov.trail_prob` (`Apn2R03/Markov.lean`)
- `APN2.Markov.round_diff` (`Apn2R03/Markov.lean`)
- `APN2.Markov.char_prob_le` (`Apn2R03/CryptoCor.lean`)
- `APN2.Markov.char_prob_le_fam3'` (`Apn2R03/ResPerm.lean`)
- `APN2.Markov.char_prob_le_fam5'` (`Apn2R03/ResPerm.lean`)

Computational cross-check: tests/test_markov_two_words.py, tests/test_crypto_c3.py.
Uses: Proposition 9.1, Theorem A.

Proposition 9.4 (p. 17): Characteristics in every 2-adic layer (Corollary G (b))

Status: formalized.
Lean:

- `APN2.Markov.char_prob_le_digit` (`Apn2R03/Layered.lean`)
- `APN2.Markov.char_prob_le_layer` (`Apn2R03/Layered.lean`)
- `APN2.GRS.top_layer_le` (`Apn2R03/Layered.lean`)
- `APN2.Markov.char_prob_le_fam5_layer` (`Apn2R03/Layered.lean`)
- `APN2.Markov.char_prob_le_top` (`Apn2R03/Layered.lean`)

Uses: Proposition 9.3, Theorem 8.4.

Proposition 9.5 (p. 17): Two-round differentials for every lift (Corollary G (c), first part)

Status: core formalized; parts on paper.

Lean:

- `APN2.Markov.hull2_le` (`Apn2R03/Hull.lean`)
- `APN2.Markov.card_out2` (`Apn2R03/HullProb.lean`)
- `APN2.Markov.diff2_prob_le` (`Apn2R03/HullProb.lean`)
- `APN2.Markov.branch_lift` (`Apn2R03/HullFam.lean`)
- `APN2.GRS.fam5_dp_le` (`Apn2R03/HullFam.lean`)
- `APN2.GRS.fam5_bij` (`Apn2R03/HullFam.lean`)
- `APN2.Markov.fam5_diff2` (`Apn2R03/HullFam.lean`)

On paper only: the passage from two to $N \geq 2$ rounds.

Computational cross-check: tests/test_hull_toy.py.

Uses: Proposition 9.1.

Proposition 9.6 (p. 18): Layerwise two-round differential bounds without a support factor (Corollary G (c), second part)

Status: core formalized; parts on paper.

Lean:

- `APN2.Markov.hull2_digit` (`Apn2R03/HullLayer.lean`)
- `APN2.Markov.diff2_digit` (`Apn2R03/HullLayer.lean`)
- `APN2.Markov.diff2_layer` (`Apn2R03/HullLayer.lean`)
- `APN2.Markov.fam5_diff2_unit_sharp` (`Apn2R03/HullLayer.lean`)
- `APN2.Markov.fam5_diff2_mid` (`Apn2R03/HullLayer.lean`)
- `APN2.Markov.diff2_top` (`Apn2R03/HullLayer.lean`)
- `APN2.Markov.hull_core` (`Apn2R03/HullUnit.lean`)
- `APN2.Markov.hull2_unit` (`Apn2R03/HullUnit.lean`)
- `APN2.Markov.diff2_unit_le` (`Apn2R03/HullUnit.lean`)
- `APN2.Markov.fam5_diff2_unit` (`Apn2R03/HullUnit.lean`)

On paper only: the passage from two to $N \geq 2$ rounds.

Computational cross-check: tests/test_hull_layer.py (GR$(4, 2)$, two words, $B = 3$, 32 permutations, all inputs and outputs).

Uses: Proposition 9.5, Theorem 8.4.

Proposition 9.7 (p. 18): Fixed keys (Corollary G (d))

Status: formalized.

Lean:

- `APN2.Markov.fixed_key_le` (`Apn2R03/HullProb.lean`)

- `APN2.Markov.fam5_fixed_key` (`Apn2R03/HullFam.lean`)
- `APN2.Markov.fam5_fixed_key_unit` (`Apn2R03/HullUnit.lean`)
- `APN2.Markov.fixed_key_layer` (`Apn2R03/HullLayer.lean`)

Uses: Proposition 9.5, Proposition 9.6.

Proposition 9.8 (p. 19): Residue partition

Status: core formalized; parts on paper.

Lean:

- `APN2.CongPart.poly_cong` (`Apn2R03/CongPart.lean`)
- `APN2.CongPart.round_cong` (`Apn2R03/CongPart.lean`)
- `APN2.CongPart.iter_cong` (`Apn2R03/CongPart.lean`)

On paper only: the comparison with a random permutation.

Theorem 10.1 (p. 19): Representatives with $\mu = 0$ and $Z = \emptyset$ for every odd m

Status: core formalized; parts on paper.

Lean:

- `APN2.Ad_inj` (`Apn2R03/RepFamily.lean`)
- `APN2.Ad_rho` (`Apn2R03/RepFamily.lean`)
- `APN2.cubeAd_inj` (`Apn2R03/RepFamily.lean`)
- `APN2.Kd_zero_iff` (`Apn2R03/RepFamily.lean`)
- `APN2.eps_not_im` (`Apn2R03/RepFamily.lean`)
- `APN2.Lam_ne_zero` (`Apn2R03/RepFamily.lean`)

On paper only: the coefficient formula of p_d and the count via Kloosterman sums.

Computational cross-check: tests/test_rep_family.py, rep_family/.

Uses: Theorem B.

Theorem 10.2 (p. 20): A sparse family (at most 16 monomials)

Status: core formalized; parts on paper.

Lean:

- `APN2.sq_add_add_one_ne` (`Apn2R03/LowCost.lean`)
- `APN2.sparse_Lam_ne` (`Apn2R03/LowCost.lean`)

On paper only: the coefficient formula of p_c and the existence proof (Bezout and Aubry-Perret).

Computational cross-check: cert/sparse_cert_r06.py (irreducibility of $t^{17} + t^3 + 1$, 125769 points over $\mathbb{F}_{2^{17}}$, counts for $m \leq 13$ by two methods); rep_sparse/.

Uses: Theorem B.

Proposition 10.3 (p. 21): Low-cost optimal lifts by one coefficient digit

Status: formalized for the family $P_{m,r}$; general case on paper.

Lean:

- `APN2.GRS.fam5'_A` (`Apn2R03/LowCost.lean`)
- `APN2.GRS.fam5'_A_eq_sharp` (`Apn2R03/LowCost.lean`)
- `APN2.GRS.fam5'_upper` (`Apn2R03/LowCost.lean`)

On paper only: the general residue p; layer optimality of digit-changed lifts; the operation count.

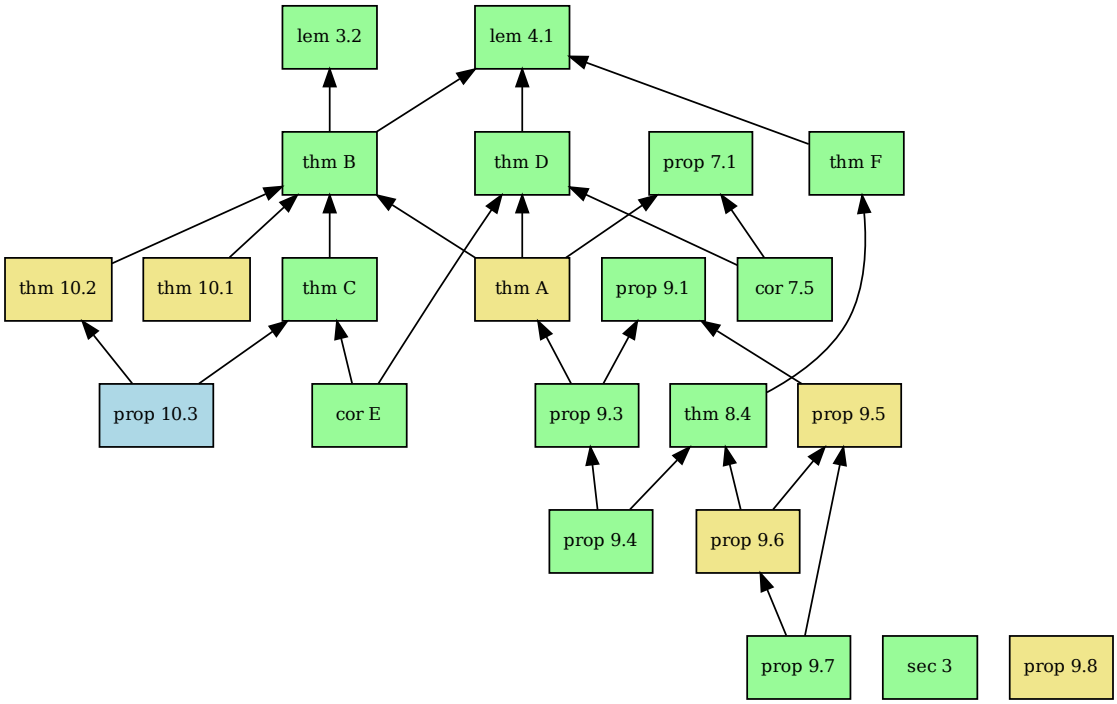
Computational cross-check: rep_sparse/.

Uses: Theorem C, Theorem 10.2.

Not formalized

The operation counts in the termwise evaluation model; the coefficient formulas and the existence and counting arguments for the representative families (Theorems 10.1 and 10.2); Proposition 8.6 and the classification of the remaining intermediate-layer configurations; the passage from two to $N \geq 2$ rounds in Propositions 9.5 and 9.6; Proposition 10.3 beyond the family $P_{m,r}$; the probability comparison in Proposition 9.8. The finite computations are replayed by `supplement/replay_core.sh` and `supplement/replay_full.sh`.

Dependency graph of the items



Import graph of the Lean modules

